

FEM PROJECT

Ruilin Zhang

HKUST

December 5, 2024

Abstract

This is a project for MATH 5311. The project contains the code, the result for a simple PDE problem solved by FEM.

1 Methodology

We construct the local stiff matrix and local F vector for each triangle element. Then we assemble the global stiff matrix and global F vector. Finally, we solve the linear system to get the solution. The code is shown in the appendix.

1.1 Local stiff matrix

For a triangle element with vertices p_1, p_2, p_3 , denote v_1, v_2, v_3 as $v_i(p_j) = \delta_{i,j}$. v_1, v_2, v_3 is the basis on p_1, p_2, p_3 respectively, which is compact supported on the triangle element.

The local stiff matrix is defined as:

$$\begin{pmatrix} \langle \nabla v_1, \nabla v_1 \rangle & \langle \nabla v_1, \nabla v_2 \rangle & \langle \nabla v_1, \nabla v_3 \rangle \\ \langle \nabla v_2, \nabla v_1 \rangle & \langle \nabla v_2, \nabla v_2 \rangle & \langle \nabla v_2, \nabla v_3 \rangle \\ \langle \nabla v_3, \nabla v_1 \rangle & \langle \nabla v_3, \nabla v_2 \rangle & \langle \nabla v_3, \nabla v_3 \rangle \end{pmatrix}$$

1.2 Global stiff matrix

We know that in local stiff function, the basis function is compact supported on the triangle element. So the inner product on local stiff matrix is not the whole inner product, we need to count the inner product in every possible triangle element.

1.3 Local F vector

local F vector is defined as:

$$\begin{pmatrix} \langle f, v_1 \rangle \\ \langle f, v_2 \rangle \\ \langle f, v_3 \rangle \end{pmatrix}$$

1.4 Global F vector

Like the argument in global stiff matrix, we need to count the inner product in every possible triangle element.

2 Result

The solver for the matrix computation is the spsolve in the python package scipy.sparse.linalg

2.1 The Error for L^2 norm

For the result matrix $\{u\}_{i,j}$, we use $\sqrt{\frac{1}{N^2} \text{Tr}((u - u_{real})(u - u_{real})^T)}$ as L^2 norm.

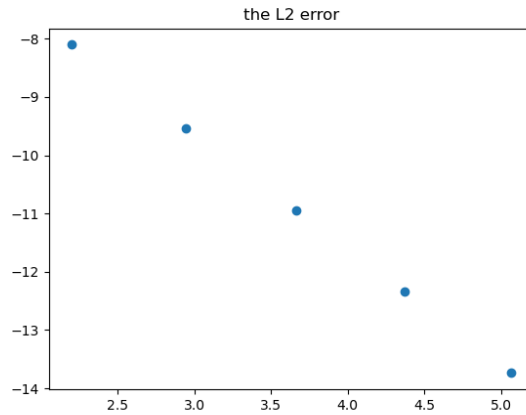


Figure 1: The L^2 norm of $u = xy(1-x)(1-y)$, no perturbation, y label is $\log(u - u_{real})$, x label is $\log(N)$

N	9	19	39	79	159
$\log(N)$	2.19722458	2.94443898	3.66356165	4.36944785	5.06890
$u = xy(1-x)(1-y)$	-8.10155806	-9.53554752	-10.94625138	-12.34489502	-13.73740
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	-8.14806774	-9.58728994	-10.99928207	-12.3982466	-13.79083

Table 1: No, perturbation. The $\log(u - u_{\text{real}})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

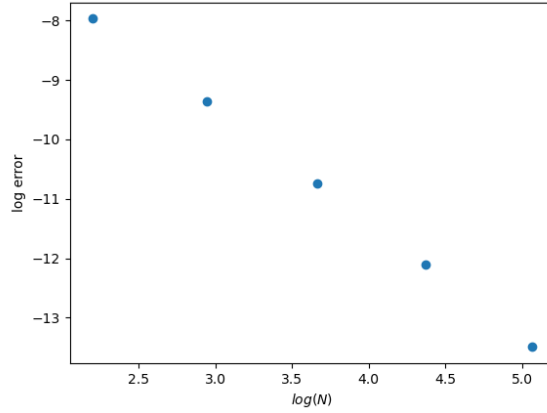


Figure 2: The L^2 norm of $u = xy(1-x)(1-y)$, with perturbation, y label is $\log(u - u_{\text{real}})$, x label is $\log(N)$

N	9	19	39
$\log(N)$	2.19722458	2.94443898	3.66356165
$u = xy(1-x)(1-y)$	-8.104678104234349	-9.519235267466975	-10.932500257838912
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	-8.13578148	-9.57542281	-10.98331034

Table 2: With perturbation. The $\log(u - u_{\text{real}})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

We have quadratic convergence rate, which is the slope of the line in the above figures. The result are consistent with the expected estimates:

$$\|e_h\|_{L^2} \leq c^2 h^2 \|u''\|_{L^2}$$

2.2 Run time

In this part, we only consider the time consumed on the solving matrix $Ku = F$.

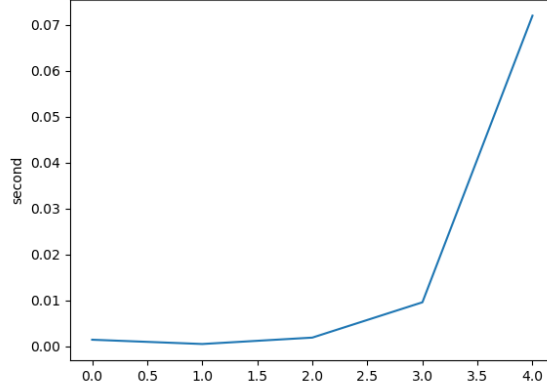


Figure 3: No perturbation, time consuming , $u = xy(1-x)(1-y)$, ylabel is time consuming(not the $\log(\text{time})$),xlabel is the $\log_2((N+1)/10)$

N	9	19	39	79	159
$\log(N)$	2.19722458	2.94443898	3.66356165	4.36944785	5.0689042
$u = xy(1-x)(1-y)$	-8.86742195	-7.76543128	-6.29792743	-4.66313349	-2.57565543
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	-6.76188562	-7.70596463	-6.24764527	-4.63704982	-2.47129635

Table 3: No perturbation. The $\log(\text{TimeConsuming})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

N	9	19	39	79	159
$\log(N)$	2.19722458	2.94443898	3.66356165	4.36944785	5.0689042
$u = xy(1-x)(1-y)$	-6.85060312	-7.66040809	-6.0281555	-4.30261561	-1.89060951
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	-6.66869447	-7.60097494	-6.00807746	-4.2916649	-2.01625672

Table 4: With perturbation. The $\log(\text{TimeConsuming})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

According to the tables, the amount of time consuming is also $O(N^2)$

2.3 Conditional number

We use the max eigenvalue/ min eigenvalue to approximate the Conditional number.

N	9	19	39	79	159
$\log(N)$	2.19722458	2.94443898	3.66356165	4.36944785	5.0689042
$u = xy(1-x)(1-y)$	3.68546007	5.08418087	6.47356504	7.86063081	9.24711796
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	3.68546007	5.08418087	6.47356504	-7.86063081	9.24711796

Table 5: No perturbation. The $\log(\text{ConditionalNumber})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

N	9	19	39	79	159
$\log(N)$	2.19722458	2.94443898	3.66356165	4.36944785	5.0689042
$u = xy(1-x)(1-y)$	3.6951828	5.08916645	6.47921456	7.86621819	9.25253407
$u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$	3.69320655	5.08908131	6.47830212	7.86607982	9.25264724

Table 6: With perturbation. The $\log(\text{ConditionalNumber})$ and the $\log(N)$ table in $u = xy(1-x)(1-y)$ and $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

The amount of Conditional number is also $O(N^2)$.

Also notice that the Conditional number for K induced by the different function is same, in the first table. Actually if the division is settled, the K is same.

3 Appendix

3.1 Code for $u = xy(1-x)(1-y)$, no perturbation, Function setting

```

1      import numpy as np
2      from scipy.sparse import lil_matrix
3      from scipy.sparse.linalg import spsolve
4
5      ## the f corespond to u=xy(1-x)(1-y) is 2(1-y)y+2(1-x)x
6      f1=lambda x,y:2*(1-x)*x+2*y*(1-y)
7      f2=lambda x,y:1
8
9      def area(x,y):
10         """

```

```

11
12     :param x: numpy x coordinate
13     :param y: numpy y coordiante
14     :return:
15     """
16     return 0.5*np.abs((x[1]-x[0])*(y[2]-y[0])-(x[2]-x[0])*(y[1]-y
17         [0]))
18
19 def inner_product(x,y,u=f1):
20     """
21     :param x: numpy x coordinate
22     :param y: numpy y coordiante
23     :param u: function
24     :return:
25     """
26     z=np.column_stack((x,y))
27     K=area(x,y)
28     s1=u(*z[1])+u(*z[2])+u(*z[0])
29     s2=u*((z[1]+z[2])/2))+u*((z[0]+z[1])/2))+u*((z[2]+z[0])/2))
30     s3=u*((z[0]+z[1]+z[2])/3))
31     return (K/60)*(3*s1+8*s2+27*s3)
32
33
34
35
36 def local_stiff(element):
37     p1,p2,p3=element
38     x=np.array([p1[0],p2[0],p3[0]])
39     y=np.array([p1[1],p2[1],p3[1]])
40     A=area(x,y)
41
42     b=np.array([p2[1]-p3[1],p3[1]-p1[1],p1[1]-p2[1]])
43     c=np.array([p3[0]-p2[0],p1[0]-p3[0],p2[0]-p1[0]])
44
45     Ke=(1/(4*A))*(b[:,None]*b[None,:]+c[:,None]*c[None,:])
46     return Ke
47
48
49 def local_F(element,f=f1):

```

```

50     p1, p2, p3 = element
51     x = np.array([p1[0], p2[0], p3[0]])
52     y = np.array([p1[1], p2[1], p3[1]])
53     #A = area(x, y)
54     A1=(p1[0]-p2[0])*(p1[1]-p3[1])-(p1[0]-p3[0])*(p1[1]-p2[1])
55     A2=(p2[0]-p3[0])*(p2[1]-p1[1])-(p2[0]-p1[0])*(p2[1]-p3[1])
56     A3=(p3[0]-p1[0])*(p3[1]-p2[1])-(p3[0]-p2[0])*(p3[1]-p1[1])
57
58
59     g1=lambda x,y:(1/A1)*(p2[0]*p3[1]-p3[0]*p2[1]+(p2[1]-p3[1])*x+(
60         p3[0]-p2[0])*y)*f(x,y)
61     g2=lambda x,y:(1/A2)*(p3[0]*p1[1]-p1[0]*p3[1]+(p3[1]-p1[1])*x+(
62         p1[0]-p3[0])*y)*f(x,y)
63     g3=lambda x,y:(1/A3)*(p1[0]*p2[1]-p2[0]*p1[1] + (p1[1] - p2[1])
64         * x + (p2[0] - p1[0]) * y)*f(x,y)
65
66     return [inner_product(x,y,g1),inner_product(x,y,g2),
67         inner_product(x,y,g3)]
68
69
70
71 if __name__=="__main__":
72     e=[[0.1,0.1],[0.1,0.2],[0.2,0.2]]
73     print(local_stiff(e))
74     print(local_F(e))

```

3.2 Code for $u = xy(1-x)(1-y)$, no perturbation, Main function

```

1     import numpy as np
2     from scipy.sparse import lil_matrix
3     from scipy.sparse.linalg import spsolve, splu, eigsh
4     from scipy.sparse.linalg import norm as snorm
5     import matplotlib.pyplot as plt
6     import matplotlib.tri as tri
7     import project_function as pf

```

```

8 import time
9
10 True_f=lambda x,y:x*y*(1-x)*(1-y)
11
12 error=[]
13 error_l2=[]
14 N=[9,19,39,79,159]
15 N_as_label=np.array(N)
16
17 time_cosume=[]
18 condition_number=[]
19
20
21 for n in N:
22     x = np.linspace(0, 1, n+2)
23     y = np.linspace(0, 1, n+2)
24     dx=dy=x[1]-x[0]
25     #print(dx)
26     X, Y = np.meshgrid(x, y)
27
28
29     tri_elements=[]
30     for i in range(n+1):
31         for j in range(n+1):
32             tri_elements.append([[i,j],[i+1,j],[i+1,j+1]])
33             tri_elements.append([[i,j],[i,j+1],[i+1,j+1]])
34
35     K=lil_matrix((n**2,n**2))
36     F=np.zeros(n**2)
37
38     # construct K
39     for element in tri_elements:
40         i1,i2,i3=element
41         p1=[X[i1[0],i1[1]],Y[i1[0],i1[1]]]
42         p2 = [X[i2[0], i2[1]], Y[i2[0], i2[1]]]
43         p3 = [X[i3[0], i3[1]], Y[i3[0], i3[1]]]
44         Ke=pf.local_stiff([p1,p2,p3])
45
46
47     for l1 in range(3):

```



```

48         if 1<=element[l1][0]<=n and 1<=element[l1][1]<=n:
49             for l2 in range(3):
50                 if 1 <= element[l2][0] <= n and 1 <= element[l2
51                     ][1] <= n:
52                     K[(element[l1][0]-1)*n+element[l1][1]-1,(
53                         element[l2][0]-1)*n+element[l2][1]-1]+=Ke
54                         [l1,l2]
55
56             #construct F
57             Fe=pf.local_F([p1,p2,p3])
58
59             for l3 in range(3):
60                 if 1<=element[l3][0]<=n and 1<=element[l3][1]<=n:
61                     F[(element[l3][0]-1)*n+element[l3][1]-1]+=Fe[l3]
62
63             K=K.tocsr()
64
65             t1=time.time()
66             u=spsolve(K,F.T).T
67             t2=time.time()
68             time_consume.append(t2-t1)
69             eigvals_max = eigsh(K, k=1, which='LM', return_eigenvectors=
70                 False)
71             eigvals_min = eigsh(K, k=1, which='SM', return_eigenvectors=
72                 False)
73             condition_number.append(eigvals_max/eigvals_min)
74
75             u=u.reshape((n,n))
76             Tu=True_f(X[1:n+1,1:n+1],Y[1:n+1,1:n+1])
77             error.append(np.max(abs(u-Tu)))
78             error_l2.append(np.linalg.norm(u-Tu)/n)
79
80 error_l2=np.array(error_l2)
81
82 plt.figure('uniform norm error')
83 plt.plot(error)
84 plt.show()
85 plt.figure()
86 plt.scatter(np.log(N_as_label),np.log(error_l2))
87 plt.xlabel('log(N)')

```

```

83 plt.ylabel('log(error)')
84 plt.title('the L2 error')
85 plt.show()
86
87
88 plt.figure('Time consuming')
89 plt.scatter(np.log(N_as_label), np.log(np.array(time_cosume)))
90 plt.ylabel('second')
91 plt.show()
92
93 print(np.log(N_as_label))
94 print('log $L^2$ error')
95 print(np.log(error_l2))
96 print('condition number')
97 print(np.log(np.array(condition_number)))
98 print('time cosume')
99 print(np.log(np.array(time_cosume)))

```

3.3 Code for $u = xy(1-x)(1-y)$, with perturbation, Main function

We only need to add the following code in each iteration.

```

1     noise1=np.random.uniform(-1,1,(n,n))
2     noise2 = np.random.uniform(-1, 1, (n, n))
3     X[1:n+1,1:n+1]+=0.05*dx*noise1
4     Y[1:n+1,1:n+1]+=0.05*dx*noise2

```

3.4 Code for $u = xy(1-x)(1-y)e^{-(x-0.5)^2-(y-0.5)^2}$

Add the following code

```

1     nabla_f=lambda x,y:-1*np.exp(-(x-0.5)**2-(y-0.5)**2)*((-2*(-x
        **2+x+1)+(1-2*x)**2*(-x**2+x+2))*(1-y)*y+(-2*(-y**2+y+1)
        +(1-2*y)**2*(-y**2+y+2))*x*(1-x))

```

```

1     #construct local F
2     Fe=pf.local_F([p1,p2,p3], nabla_f)

```

when construct local F matrix.